

# Exam Projects for Deep Learning for PDEs in Engineering Physics

July 1, 2026

## Problem A: Recover Young's modulus in 1D Elastostatics

Consider a rod made of linearly elastic material subjected to some load. Static problems will be considered here, by which is meant it is not necessary to know how the load was applied, or how the material particles moved to reach the stressed state; it is necessary only that the load is applied slowly enough so that the accelerations are zero, or that it was applied sufficiently long ago that any vibrations have died away and movement has ceased.

The equation governing the static response of the rod is:

$$-\frac{d}{dx} \left( k(x) \frac{du}{dx} \right) = f, \quad x \in (0, L) \quad (1)$$

where

- $u(x)$  : displacement field of the rod
- $k(x)$  : Young's modulus
- $f = 9.81$  : body force per unit length (e.g., gravity)
- $L = 1$  : Length of the rod

We consider the fixation of both sides of the rod, which leads to the following boundary conditions:

$$u(0) = u(L) = 0 \quad (2)$$

### Task: Recover the Young's modulus $k(x)$ from the observation of displacement field $u(x)$

In this task, the Young's modulus  $k(x) > 0$  of the rod is unknown. However, we observe the displacement field  $u_{obs}$  (contaminated by noise with a noise level  $\sim 5\%$ ) on a set of randomly placed sensors  $x_{obs}$  (with size  $N_{obs} = 500$ ).

#### The goals

- Please select a suitable deep learning method for solving this inverse problem to recover the Young's modulus  $k(x)$ , and explain the reason for using it
- Report your setups for the implementation, such as network structure, activation function, optimizer (with learning rate), epoch (with batch size), loss weights, and other tricks that are used for improvement.
- Compute the  $L^2$  relative error (on the testing dataset) at each training epoch and plot the **Error vs. epoch** curve (and report the final error). The  $L^2$  relative error between the prediction  $k_{pred}$  (or  $u_{pred}$ ) and the truth  $k_{true}$  (or  $u_{true}$ ) is defined as follows:

$$error = \sqrt{\frac{\sum_i^n |k_{pred}(x_i) - k_{true}(x_i)|^2}{\sum_i^n |k_{true}(x_i)|^2}}$$

- Plot the predicted solution (and the ground truth reference) and the pointwise absolute error using separate figures with `matplotlib`.

## Dataset

The ground truth reference is given in the dataset `ProblemA_dataset.h5`:

- `x_obs`: the observation sensors
- `u_obs`: the observed displacement field  $u$  (contaminated by noise)
- `x_test`: the locations where the ground truth is evaluated (Used for computing error and should **not** be used for training)
- `k_test`: the ground truth reference for Young's modulus (Used for computing error and should **not** be used for training)
- `u_test`: the ground truth reference for the displacement field (Used for computing error and should **not** be used for training)
- Link to dataset: <https://www.kaggle.com/datasets/yhzang32/dno4pdes>

## Problem B: Solve Pressure Field in 2D Heterogeneous Porous Media

Consider a square domain  $\Omega = [0, 1]^2$  representing a heterogeneous porous medium composed of two distinct material phases with significantly different permeabilities, for instance, high-permeability sandstone inclusions embedded in a low-permeability mudstone matrix. Such heterogeneous structures arise frequently in subsurface flow engineering, including groundwater modeling, hydrocarbon reservoir simulation, and fuel cell design. Near the interface between the two phases, the pressure gradient exhibits sharp transitions, making this a classical benchmark for testing the robustness of PDE solvers under non-smooth coefficients.

The permeability field  $\mu(x)$  is defined as:

$$\mu(x) = \begin{cases} \mu_1 = 10, & x \in \Omega_1 \quad (\text{high-permeability phase, e.g., sandstone}) \\ \mu_2 = 2, & x \in \Omega_2 \quad (\text{low-permeability phase, e.g., mudstone}) \end{cases}$$

The specific two-phase microstructure (i.e., the geometric distribution of  $\Omega_1$  and  $\Omega_2$ ) is provided in the dataset. The pressure field  $u(x)$  satisfies the following Darcy flow equation:

$$-\nabla \cdot (\mu(x) \nabla u) = f, \quad x \in \Omega \quad (3)$$

where

- $u(x)$ : pressure field in the porous medium
- $\mu(x)$ : spatially varying permeability field
- $f = 0$ : source term (no internal source/sink)
- $\Omega = [0, 1]^2$ : square computational domain

The boundary conditions are prescribed as follows, driving fluid flow from the high-pressure inlet on the left to the low-pressure outlet on the right:

$$g_D(x) = \begin{cases} 1, & x_1 = 0 \quad (\text{left boundary, high-pressure inlet}) \\ 0, & x_1 = 1 \quad (\text{right boundary, low-pressure outlet}) \\ \text{linear interpolation,} & x_2 = 0 \text{ or } 1 \quad (\text{top and bottom boundaries}) \end{cases}$$

### Task: Solve the pressure field $u(x)$

In this task, the permeability field  $\mu(x)$  is provided as a  $128 \times 128$  pixel matrix — that is, the domain is subdivided into  $128 \times 128$  cells, each with a constant permeability value. Using this permeability field, apply a suitable method introduced in the lecture to solve for the pressure field  $u(x)$  in the heterogeneous porous medium.

### The Goals

- Please select a suitable deep learning method for solving this forward problem to compute the pressure field  $u(x)$ , and explain the reason for using it.
- Report your setups for the implementation, such as network structure, activation function, optimizer (with learning rate), epoch (with batch size), loss weights, and other tricks that are used for improvement.

- Compute the  $L^2$  relative error (on the testing dataset) at each training epoch and plot the **Error vs. epoch** curve (and report the final error). The  $L^2$  relative error between the prediction  $u_{\text{pred}}$  and the reference  $u_{\text{true}}$  is defined as follows:

$$\text{error} = \sqrt{\frac{\sum_{i=1}^n |u_{\text{pred}}(x_i) - u_{\text{true}}(x_i)|^2}{\sum_{i=1}^n |u_{\text{true}}(x_i)|^2}}$$

- Plot the predicted solution (and the ground truth reference) and the pointwise absolute error using separate figures with `matplotlib`.

## Dataset

The permeability field and the reference solution are provided in the dataset `ProblemB_dataset.h5`:

- `mu_field`: the permeability field  $\mu(x)$ , represented as a  $128 \times 128$  pixel matrix, where each cell has a constant permeability value (available for training)
- `x_test`: the locations where the reference solution is evaluated (used for computing error and should **not** be used for training)
- `u_test`: the reference pressure field values (used for computing error and should **not** be used for training)
- Link to dataset: <https://www.kaggle.com/datasets/yhzang32/dno4pdes>

## Problem C: Surrogate Modeling for Steady-State Heat Conduction in Heterogeneous Materials

Consider the steady-state heat conduction problem in a heterogeneous solid material occupying the unit square domain  $\Omega = [0, 1]^2$ :

$$\begin{aligned} -\nabla \cdot (a(x, y) \nabla u) &= f, & (x, y) \in \Omega &= [0, 1]^2 \\ u &= 0, & (x, y) \in \partial\Omega \end{aligned} \tag{4}$$

where  $u(x, y)$  denotes the temperature field,  $a(x, y) > 0$  denotes the spatially varying thermal conductivity of the material, and the uniform heat source is given by  $f = 10$ . The zero Dirichlet boundary condition models a configuration in which the boundary of the domain is kept at a fixed reference temperature.

In computational materials science, evaluating the thermal response of a large number of candidate microstructures is a central bottleneck in material screening and design workflows. Although high-fidelity Finite Element Method (FEM) solvers can produce accurate solutions, they are computationally expensive when applied repeatedly to thousands of different conductivity fields  $a(x, y)$ .

### Task: Learning the Solution Operator $a(x, y) \mapsto u(x, y)$

The goal of this task is to make a fast prediction of the temperature field  $u(x, y)$  given  $a(x, y)$ . To this end, a dataset of input-output pairs  $\{a^{(j)}, u^{(j)}\}$  has been pre-computed using a high-precision FEM solver. The conductivity fields  $a^{(j)}$  are sampled from a distribution  $\mathcal{A}$  over spatially heterogeneous functions, representative of realistic microstructural variability in composite materials. Once trained, the model should accurately predict the temperature field for new, unseen conductivity samples drawn from the same distribution  $\mathcal{A}$ , at a fraction of the cost of a full FEM solve.

### Goals

- Please select a suitable deep learning method for this task and justify your choice.
- Report your implementation setup, including network architecture, activation function, optimizer (with learning rate), number of epochs, batch size, loss formulation, and any additional techniques used for improvement.
- Compute the  $L^2$  relative error on the **test dataset** at each training epoch and plot the **Error vs. Epoch** curve (reporting the final error). The  $L^2$  relative error between predictions  $\{u_{\text{pred}}^{(j)}\}_{j=1}^N$  and ground truth  $\{u_{\text{true}}^{(j)}\}_{j=1}^N$  is defined as:

$$\text{error} = \frac{1}{N} \sum_{j=1}^N \sqrt{\frac{\sum_i |u_{\text{pred}}^{(j)}(x_i) - u_{\text{true}}^{(j)}(x_i)|^2}{\sum_i |u_{\text{true}}^{(j)}(x_i)|^2}}$$

where  $j$  indexes the sample and  $i$  indexes the spatial grid point.

- For the **first test instance**, use `matplotlib` to plot the following in separate figures:

1. The input conductivity field  $a(x, y)$
2. The predicted temperature field  $u_{\text{pred}}(x, y)$
3. The ground truth temperature field  $u_{\text{true}}(x, y)$
4. The pointwise absolute error  $|u_{\text{pred}} - u_{\text{true}}|$

## Dataset

The training and test data are provided in `ProblemC_dataset.h5`:

| Key                  | Shape            | Description                                                                                     |
|----------------------|------------------|-------------------------------------------------------------------------------------------------|
| <code>a_train</code> | (1000, 128, 128) | Training conductivity fields sampled from $\mathcal{A}$                                         |
| <code>u_train</code> | (1000, 128, 128) | Corresponding FEM-computed temperature fields (training)                                        |
| <code>a_test</code>  | (200, 128, 128)  | Test conductivity fields (for error evaluation only, <b>do not use for training</b> )           |
| <code>u_test</code>  | (200, 128, 128)  | Corresponding FEM-computed temperature fields (test reference, <b>do not use for training</b> ) |
| <code>X</code>       | (128, 128)       | $x$ -coordinates of the $128 \times 128$ regular spatial grid                                   |
| <code>Y</code>       | (128, 128)       | $y$ -coordinates of the $128 \times 128$ regular spatial grid                                   |

Dataset download: <https://www.kaggle.com/datasets/yhzang32/dno4pdes>

## Problem D: Prediction of Traffic Flow Based on Burgers' Equation Model

Burgers' equation is a classical PDE model that captures the interplay between nonlinear convection and diffusion, making it well-suited for describing traffic flow dynamics. In this context, the convective term models the tendency for traffic to compress as vehicle density increases, while the diffusive term reflects the smoothing effect of individual driver reactions to surrounding traffic conditions.

The viscous Burgers' equation governing the evolution of vehicle velocity is:

$$u_t + u u_x = \nu u_{xx}, \quad x \in (-1, 1), \quad t \in (0, 1] \quad (5)$$

where:

- $u(x, t)$ : vehicle velocity field (m/s),
- $\nu = 0.1/\pi$ : kinematic viscosity coefficient (a higher value reflects more cautious, diffusive driver behavior),
- $x$ : position along the road segment,
- $t$ : time.

Homogeneous Dirichlet boundary conditions are imposed at both ends of the road:

$$u(-1, t) = u(1, t) = 0, \quad t \in (0, 1]$$

Given an initial velocity profile  $u(x, 0) = a(x)$ , the full spatio-temporal velocity field  $u(x, t)$  for  $t > 0$  is uniquely determined by solving Eq. (5). In practice, however, running a high-fidelity finite difference solver for every new initial condition is computationally prohibitive. The goal of this task is therefore to learn a fast surrogate that approximates the solution operator  $a(x) \mapsto u(x, t)$ .

### Task: Predicting the Velocity Field $u(x, t)$ Given the Initial Field $a(x)$

A dataset of initial conditions  $a(x)$  sampled from a distribution  $\mathcal{A}$  has been collected, along with the corresponding velocity fields  $u(x, t)$  computed by a high-precision Finite Difference Method (FDM) solver. The dataset is **partially labeled**: only a small subset of initial conditions is paired with its FDM solution, while a much larger set of initial conditions has no corresponding solution available. This reflects a realistic scenario in which high-fidelity simulation data is scarce and expensive to obtain. The trained surrogate model should accurately predict  $u(x, t)$  for any new initial condition  $a(x)$  drawn from the same distribution  $\mathcal{A}$ , at negligible computational cost.

### Goals

- Select a suitable deep learning method for this task and justify your choice.
- Report your implementation setup, including network architecture, activation function, optimizer (with learning rate), number of epochs, batch size, loss formulation, and any additional techniques used for improvement.



- Compute the  $L^2$  relative error on the **test dataset** at each training epoch and plot the **Error vs. Epoch** curve (reporting the final error). The error is defined as:

$$\text{error} = \frac{1}{N} \sum_{j=1}^N \sqrt{\frac{\sum_i |u_{\text{pred}}^{(j)}(x_i, t_i) - u_{\text{true}}^{(j)}(x_i, t_i)|^2}{\sum_i |u_{\text{true}}^{(j)}(x_i, t_i)|^2}}$$

where  $j$  indexes the sample and  $i$  indexes the spatio-temporal grid point.

- For the **first test instance**, use `matplotlib` to produce four separate figures:
  1. The initial condition  $a(x) = u(x, 0)$
  2. The predicted velocity field  $u_{\text{pred}}(x, t)$
  3. The ground truth velocity field  $u_{\text{true}}(x, t)$
  4. The pointwise absolute error  $|u_{\text{pred}} - u_{\text{true}}|$

## Dataset

All data are provided in `ProblemD_dataset.h5`:

| Key                            | Shape           | Description                                                                           |
|--------------------------------|-----------------|---------------------------------------------------------------------------------------|
| <code>a_train_labeled</code>   | (200, 256)      | $N = 200$ <b>labeled</b> initial fields, sampled at 256 spatial sensors               |
| <code>u_train_labeled</code>   | (200, 200, 256) | Corresponding FDM-computed velocity fields on a $200 \times 256$ spatio-temporal grid |
| <code>a_train_unlabeled</code> | (1800, 256)     | $N = 1800$ <b>unlabeled</b> initial fields (no paired solution available)             |
| <code>a_test</code>            | (200, 256)      | Test initial conditions ( <b>do not use for training</b> )                            |
| <code>u_test</code>            | (200, 200, 256) | Ground truth velocity fields for test instances ( <b>do not use for training</b> )    |
| <code>x_mesh</code>            | (256, 1)        | Spatial coordinates of the 256 grid points                                            |
| <code>t_mesh</code>            | (200, 1)        | Temporal coordinates of the 200 time steps                                            |

Dataset download: <https://www.kaggle.com/datasets/yhzang32/dno4pdes>

# About the Project Report

You are asked to report your experiments. You are given a downloadable [L<sup>A</sup>T<sub>E</sub>X template<sup>1</sup>](#), which you can edit using TUMShareLatex. The report should contain the following sections:

1. **Title page** including name and student ID number
2. **Method** (max. three pages)
  - Establishing consistent symbols
  - Mathematical formulation of both network structures
  - Introduce the PDE and the loss
  - Theoretical advantages and disadvantages of both structures
  - Establish metrics for comparisons in the results part
  - Cite the relevant works
3. **Results**
  - Problem setup, used optimization methods (and parameters), and used hardware
  - Exact architecture of the used neural networks (layers, neurons, activation functions, etc.)
  - Presentation of the results of the different numerical experiments
  - Interpretation of the results
4. **Conclusion**
5. **References**: list of all sources cited
6. **Appendices** (if needed): contains lengthy materials, procedures, tables, or figures

## Important Notes

- You must provide a link to the original code you wrote for solving the PDE problems. This can be hosted on platforms such as GitHub, Google Colab, or Kaggle Notebooks. If possible, please include training logs or experiment outputs to support reproducibility.
- **You are not allowed to use pre-built implementations of deep learning methods** (e.g., PINNs, FNOs, etc.) from open-source packages or libraries. All key components must be implemented by you unless otherwise permitted in class.
- **The use of ChatGPT, Copilot, or any other AI coding assistants is strictly prohibited.** All code and analysis must reflect your own independent work.

---

<sup>1</sup>Please do not edit the original template.